

```
// josa.bootcamp / 2026 / week_01
const lecture = session.init({
  track:    "AI & Product",
  duration:"3h",
  mode:     "lecture + lab"
});
```

AI & The New Product Era.

How AI is reshaping products, users, and the industry.

PRESENTER

Yazan Shannak

AI Manager at Revest • MSc Data
Science at JUST

PREVIOUSLY

**Data Scientist at
ArabiaWeather**

ML Engineer at Tarjama •
Software Developer

CONTEXT

**JOSA AI
Bootcamp 2026**

Week 1 • Lecture + Lab

// WEEK_01

01

/ 6 WEEKS



// YOUR PRESENTER

Yazan Shannak.

AI Manager at **Revest** • MSc Data Science at **JUST**

// CURRENTLY

AI Manager

Revest • 2024 – present

// PREVIOUSLY

Data Scientist

ArabiaWeather • ML Eng. at Tarjama

// STUDYING

MSc Data Science

Jordan University of Science & Tech

////////////////////////////////

// SESSION OUTLINE

Today's agenda.

// PART 01 • THE LANDSCAPE

01 Why is AI important now?

02 Hype vs. Reality

03 What are AI agents?

// PART 02 • BUILDING

04 ⚡ Lab: build a personal expense tracker with an AI agent

////////////////////

// 01 / THE LANDSCAPE • WHY NOW?

AI adoption is **massive** — and accelerating.

People are adapting to a new expectation of **"intelligent" machines**. What was a niche research demo three years ago is now mainstream infrastructure used by billions of people every week.

800M+

ChatGPT weekly active users • early 2026
(2× growth in 12 months)

2B

Monthly users seeing Google AI Overviews
(inside Search, every search)

28%

U.S. employed adults using ChatGPT
for work tasks (Pew Research)

01

The pace is **unprecedented.**

Research, investment, and deployment are all compounding at once. **What was impossible in 2022 is a \$0.001 API call today.** The models improve faster than anyone's roadmap.

\$2.5T

Worldwide AI spending forecast for 2026 (Gartner)

10x

Drop in inference cost per token over the past 18 months

~6 mo

Average gap between SOTA model and an open-weights equivalent

02

////////////////////

// 01 / THE LANDSCAPE • HYPE VS REALITY

Hype vs. reality — claim 01.

// THE CLAIM

"AGI and ASI are right around the corner."

HYPE

Vague, moving goalposts. We're **incrementally improving models** and the harnesses around them — there's no clear threshold where AGI begins.

"AGI" has been redefined a dozen times since 2022. Every benchmark hit becomes "not really AGI." Useful framing: don't wait for a magic threshold. Ask what the model can do *today* and build for that.

////////////////////
// 01 / THE LANDSCAPE • HYPE VS REALITY

Hype vs. reality – claim 02.

// THE CLAIM

"AI will take all our jobs."

NUANCED

Real, but not as widespread as claimed. There is even the argument that we will see **more people in some AI enabled fields**, not fewer – productivity per person rises and the field grows.

Customer support, copywriting, and translation are being reshaped right now. But the bottleneck shifts to judgement, taste, and orchestration – work that humans still do best. The risk is for people who refuse to use the tools, not for whole categories of work.

////////////////////
// 01 / THE LANDSCAPE • HYPE VS REALITY

Hype vs. reality — claim 03.

// THE CLAIM

"AI will replace software developers."

MIX

Developers are being redefined, not replaced. The new developer uses AI agents to write software that is still **maintainable, of quality, and good taste**. That developer will be here. Old-style solo coders — less so. Mass Layoffs are not solely dependent on AI adoption

GitHub data shows code generated by AI assistants has crossed 30% on assisted projects. The skill that matters now is system design, code review, debugging at the boundary of what the agent can't see, and product judgement — not raw typing speed.

////////////////////
// 01 / THE LANDSCAPE • HYPE VS REALITY

Hype vs. reality – claim 04.

// THE CLAIM

"Google Search is dead. Product X is finished."

HYPE

Google Search hit **\$63B in Q4 2025 revenue – up 17% YoY**. Products evolve. They rarely vanish. AI Overviews now reach **2 billion monthly users inside Search**.

Lesson: "X-killer" headlines are almost always wrong. Incumbents with distribution, brand, and data adapt – they don't get displaced by a chat box. The real disruption is products built natively around agents, not feature swaps inside legacy ones.

////////////////////

// 01 / THE LANDSCAPE

What good looks like.

Products that rebuilt around agents — with real results.

Ramp

// corporate finance → agentic financial ops

\$1B+ ARR

\$1 billion in annualized revenue by late 2025 — doubled in one year. Now valued at \$32B, seeking \$40B.

- 26M AI decisions/month across \$10B in spend
- Fraud agent blocked a \$49,000 AI-generated fake invoice
- Companies on Ramp spend 5% less, grow 12% faster

Google Search

// keyword search → AI-native answer engine

\$63B Q4 Revenue

Search revenue up 17% YoY in Q4 2025. AI Overviews now appear on ~48% of all queries, serving 2B monthly users.

- AI Mode: 100M+ MAU, 1B+ monthly queries
- AI Mode queries are 3× longer than traditional
- GenAI products grew ~400% in revenue (Q4 2025)

ChatGPT

// chatbot → default thinking tool for millions

800M weekly users

800M weekly active users by early 2026. Doubled from 400M in Feb 2025. 5th most visited site on the internet.

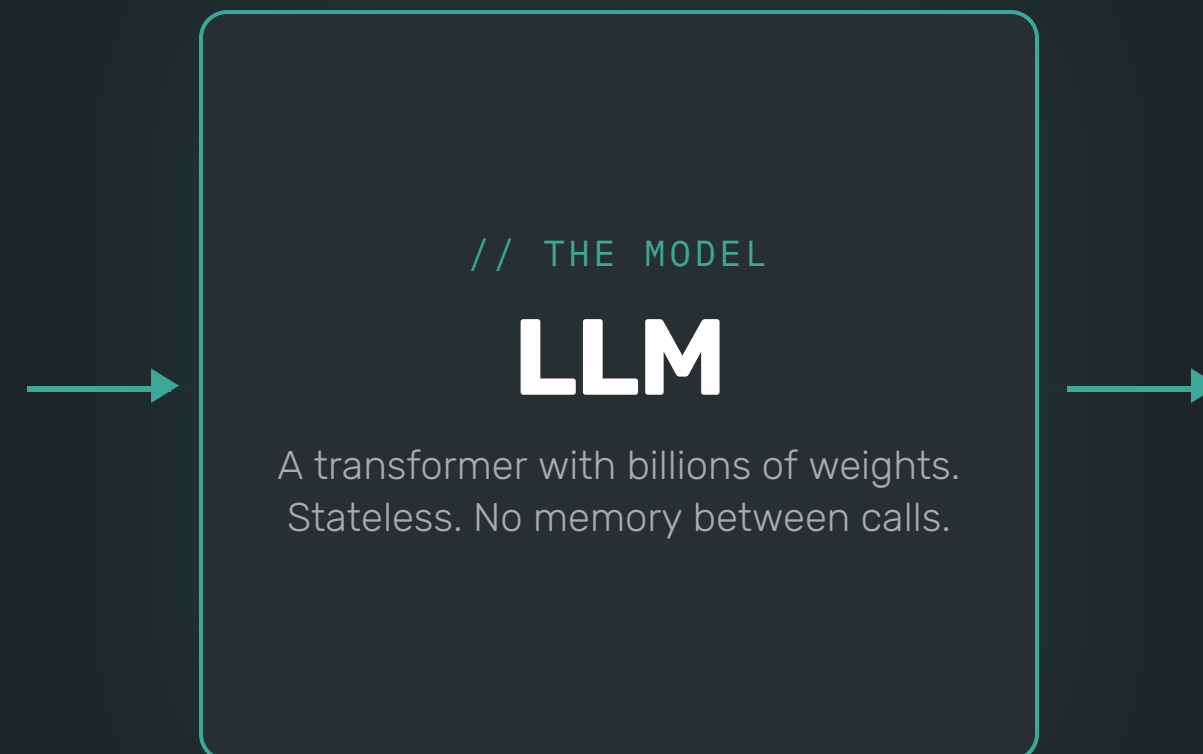
- 2 billion queries processed every single day
- 28% of U.S. employed adults use it for work
- OpenAI projecting \$29.4B in 2026 revenue

// Part 01 · done → Part 02 · starting

So what actually is an **AI agent**?

Let's build it up step by step — from a passive language model to an agent that acts in the world.

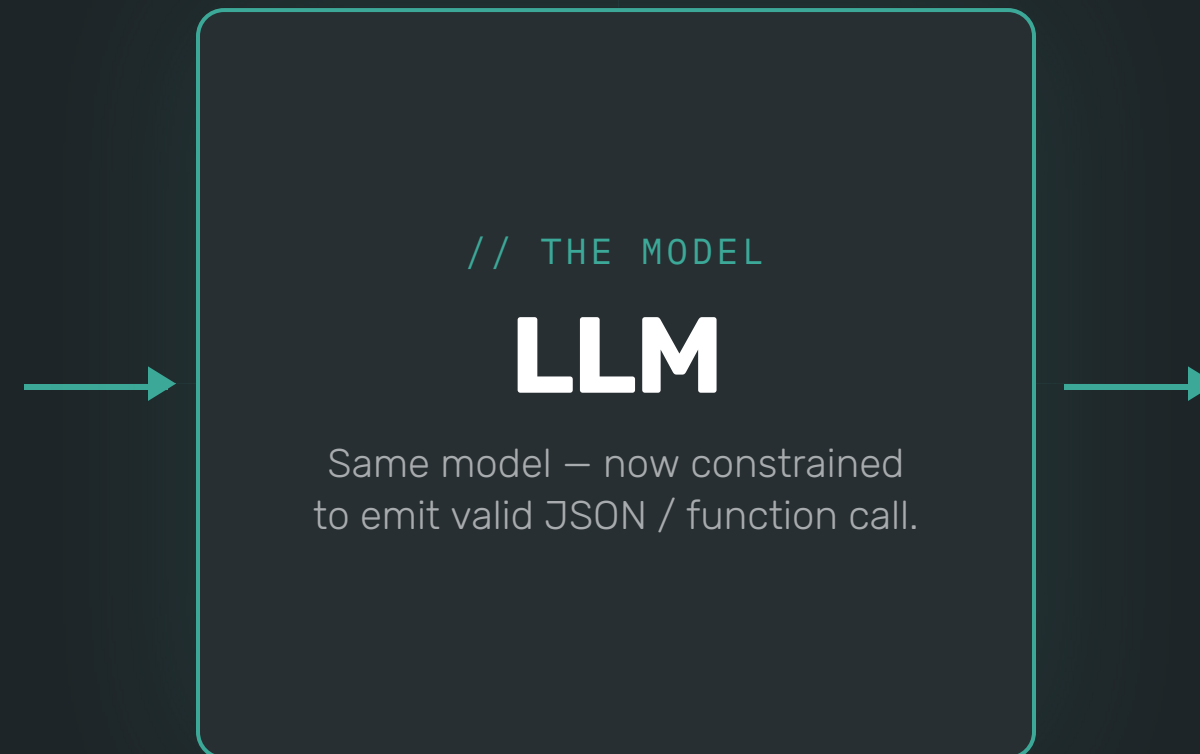
Step 01 – the basic LLM.



// THE TAKEAWAY

Text in, text out. Impressive – but passive. The LLM can't take any action, see anything outside its training, or remember our last conversation.

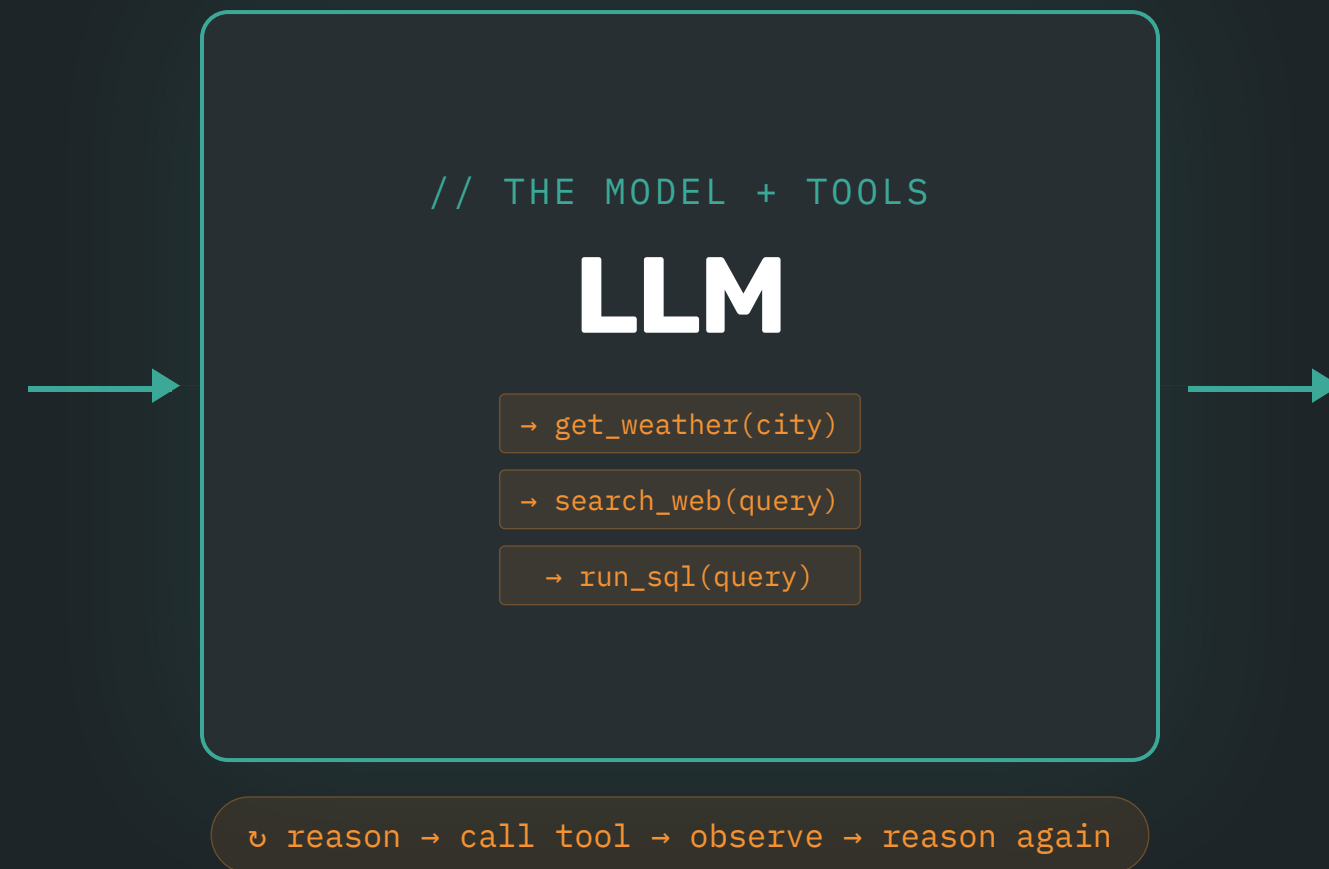
Step 02 – structured output.



// THE TAKEAWAY

Now we can route the output to real code. JSON, function calls, schemas. The LLM stops being a chat box and becomes a programmable component.

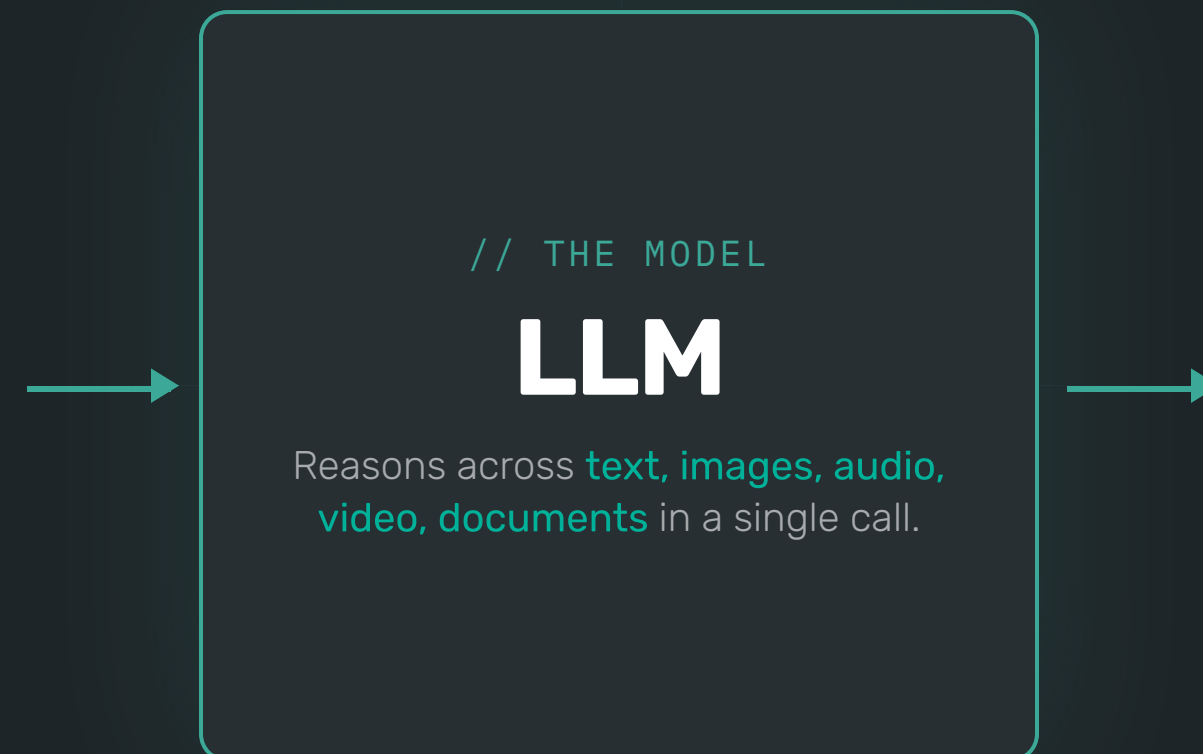
Step 03 – native tool use.



// THE TAKEAWAY

The LLM decides which tool to call, invokes it, observes the result, and keeps reasoning. This is the basic agent loop. The model is no longer locked inside its training data.

Step 04 – multimodal input.

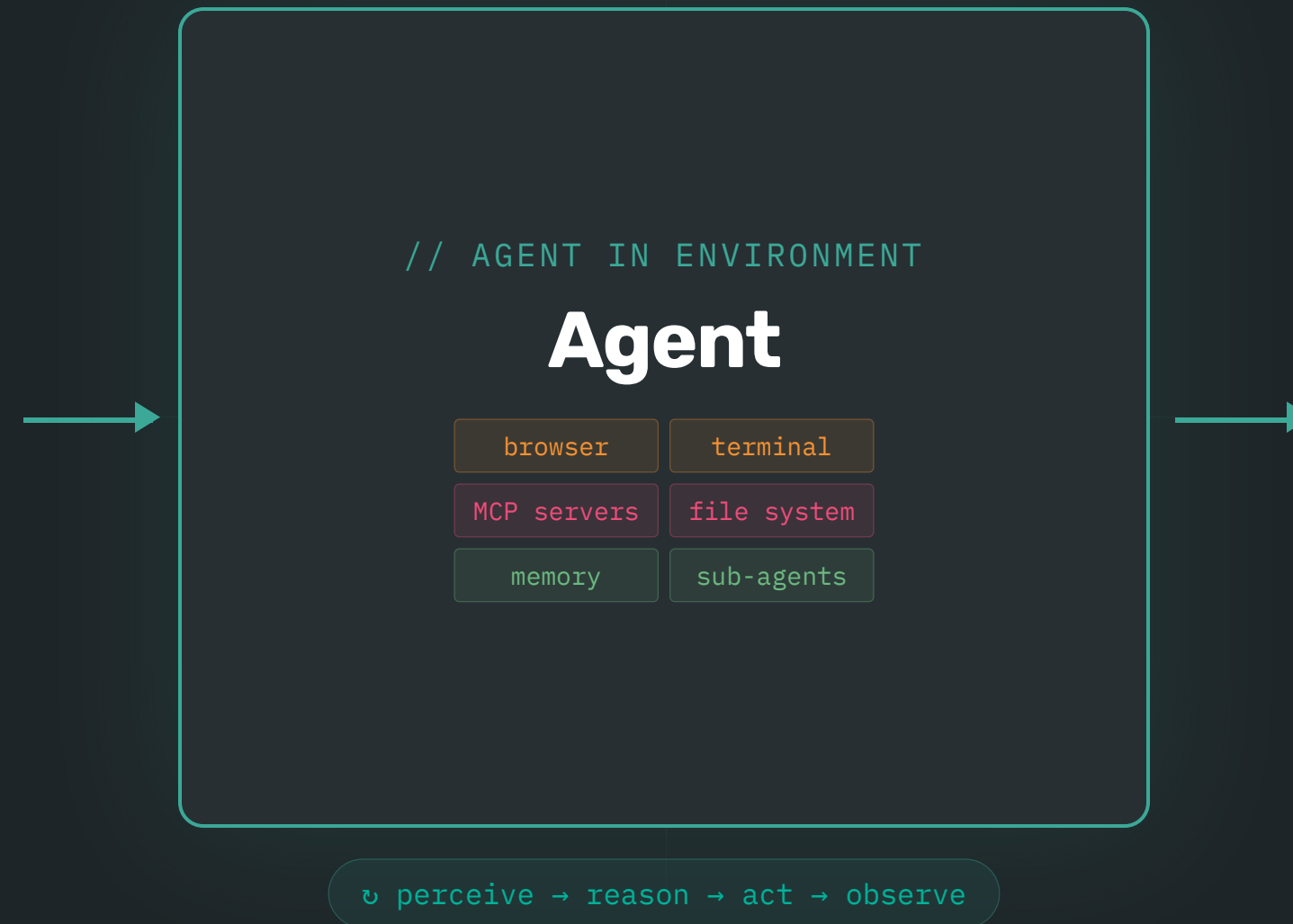


// THE TAKEAWAY

The world isn't just text. Receipts, voice notes, screenshots, PDFs – modern LLMs ingest them all. This is what unlocks "describe what you want, in any form" interfaces.

////////////////////
// 02 / WHAT ARE AGENTS? · STEP 05 OF 05 · TODAY

Step 05 – agents with environments.



// WHERE WE ARE TODAY

Agents live in and act on the world. Sandboxes, browsers, terminals, MCP servers, multi-agent pipelines. This is the substrate the rest of the lecture is about.

// WEEK 04 / LECTURE

////////////////////////////////

// 02 / WHAT ARE AGENTS? • ANATOMY 01 OF 02

An agent has two parts. First — the **LLM**.

The reasoning engine.

Takes context in, produces text or structured output.
Stateless — it has no memory of its own across calls.
Everything it "knows" lives in two places.

// weights	Billions of parameters trained on human text. Frozen at training time. The "knowledge" lives here.
// context	Everything the model sees right now — your messages, tool results, memory, system prompt. Limited window.
// output	Text, JSON, or a function call. The next layer (the harness) decides what to do with it.

// THE MODEL

LLM

```
0.3421 0.0017 -0.4421 0.9012 0.1188 ...  
... -0.2210 0.0884 0.6441 0.0029 0.7773  
0.4456 -0.0091 0.3320 0.5512 -0.1104 ...
```

~ 70 billion parameters • stateless • deterministic

Second – the **harness**. Everything that wraps the LLM.

Each piece feeds into the model's context. Swap any of them and you get a different agent.

// SYSTEM PROMPT

The instructions

Persona, rules, format, domain knowledge. Your invariant message to the model.

// USER INPUT

The request

What the user just asked, plus the running conversation history.

// TOOLS

The capabilities

Functions the model can call – DB queries, APIs, web search, code execution.

// LLM

Agent

reads everything around → emits text or tool call

// MCPS

External services

Model Context Protocol – standardized connectors to data sources and apps.

// OUTPUT

The action

Text reply, structured JSON, or a tool call the harness then executes.

// MEMORY

What it remembers

Short-term in context • long-term in vector DB / files. Survives across sessions.

////////////////////

// 02 / WHAT ARE AGENTS? • MODEL PROVIDERS 01 OF 02

Where the LLM comes from — option 01.

Closed-weights models. API only. Top-tier quality, lowest control.

Closed **weights**.

// API-ONLY • PROPRIETARY MODELS

- + **Generally highest quality.** Best reasoning, instruction following, tool use.
- + **Zero infrastructure** — call an API, get a response. No GPUs to manage.
- **API-only.** No local inference, no access to raw weights.
- **Per-token pricing** — expensive at scale.
- **Privacy risk** — provider sees your prompts; some retain data.

// THE MAJOR PLAYERS



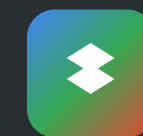
OpenAI

GPT-5 • GPT-4.1 • o3 / o4-mini



Anthropic

Claude 4 Opus • Sonnet • Haiku



Google

Gemini 2.5 Pro • Flash • Ultra



xAI

Grok-4 • Grok-3

// BEST FOR
// WEEK 01 / LECTURE

Production apps • complex reasoning • rapid prototyping

////////////////////

// 02 / WHAT ARE AGENTS? • MODEL PROVIDERS 02 OF 02

Where the LLM comes from – option 02.

Open-weights models. Run them yourself. The gap with closed models is closing fast.

Open weights.

// SELF-HOSTABLE • DOWNLOADABLE MODELS

- + Increasingly competitive – **gap with closed models is narrowing** every month.
- + **Run locally** – your data never leaves your machine.
- + **Finetunable** – adapt to your domain, your data, your style.
- + **Available via cheap APIs too** (Groq, Together, Fireworks).
- You manage the inference stack – GPUs, batching, scaling.

// THE MAJOR PLAYERS



Meta • Llama

Llama 4 • Llama 3.3 70B



Alibaba • Qwen

Qwen 3 • Qwen 2.5 Max • Coder



Mistral AI

Mistral Large • Mixtral • Codestral



DeepSeek

DeepSeek R2 • V3 • Coder V2

// BEST FOR

Privacy-sensitive apps • cost optimization • customization

// WEEK_01 / LECTURE

////////////////////

// 02 / WHAT ARE AGENTS? · BUILDING HARNESSES 01 OF 02

How you build the harness — option 01.

Low-level SDKs. Raw building blocks. You wire the loop yourself.

Building blocks.

// LOW-LEVEL SDKS · MAXIMUM CONTROL

Give you raw primitives — send messages, parse outputs, register tools. **Maximum control, higher complexity.** You decide the agent loop, error handling, retries, schemas.

- Use when you need full control over context, retries, streaming.
- Best for production systems with custom requirements.
- Steeper learning curve — but you understand every byte.

// THE TOOLKIT



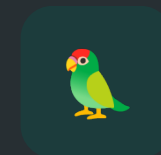
pydantic-ai

Type-safe, structured output. Pythonic, testable, model-agnostic. Best for production.



openai-sdk · anthropic-sdk

Raw API access. No opinions. Direct mapping to provider features. Use to learn what's actually happening.



langchain · langgraph

Graph-based agent workflows. Powerful for complex multi-step chains. Heavy abstractions.



vercel ai-sdk

TypeScript-first. Streaming, tool calls, React hooks built in. Best for web apps.

////////////////////

// 02 / WHAT ARE AGENTS? • BUILDING HARNESSES 02 OF 02

How you build the harness — option 02.

Pre-built coding agents. The loop is solved. You direct, you don't wire.

Opinionated agents.

// PRE-BUILT • AGENTIC BY DEFAULT

Pre-built for software engineering tasks. **The agent loop, tool calling, context management — already handled.**

You direct in plain English. The agent reads your repo, writes code, runs tests, fixes errors.

- Use when you need to ship software fast, not study the agent loop.
- Best for prototyping, glue code, ops tasks, refactors.
- Less control — but the gap closes every release.

// THE PLAYERS



Claude Code

Agentic coding in terminal. Reads, writes, runs, tests your codebase autonomously. By Anthropic.



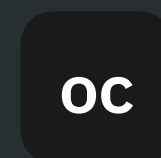
Codex CLI

Natural language → shell commands and code. Sandboxed by default. By OpenAI.



Cursor · Windsurf

AI-native IDEs. Inline edits, tab completion, repo-aware chat. Where most devs live now.



opencode · aider · pi

Open-source community alternatives. Customizable, model-agnostic. Run any provider you like.

// UP NEXT • LAB SESSION

Now let's build it.

Take everything we just covered and ship a working agent in a real product.

// THE PROJECT

A **personal expense tracker** — a simple CRUD app with transactions, categories, and a balance view. A classic system of records every household needs.

// THE CHALLENGE

Add an **AI agent layer**. Not a chat wrapper — a real agent that **reads** your statements, **categorizes** spending, and **advises** you in natural language.

// WHAT YOU'LL LEARN

System prompts • tool design • structured output • context management • evaluation. Everything from the lecture, applied to a product you'd actually ship.

\$ Expense Tracker

// week_01 lab project

Balance • this month

JOD **1,284.50**

	Carrefour City Mall Groceries • Food	-42.20
	Orange Jordan Telecoms • Bills	-28.00
	Careem Transport	-6.75
	Salary — May Income	+1,400.00

💡 You spent **18% more on groceries** than last month. Want me to set a JOD 320 cap?

// WEEK_01 / FIN

See you in the lab.

Bring a laptop, a question, and one product idea you'd build with an agent today.



// workshop repo
[github.com/YazanShannak/
josa-ai-bootcamp-demo](https://github.com/YazanShannak/josa-ai-bootcamp-demo)

// APPENDIX • SOFTWARE ENGINEERING BASICS

Background reading.

Quick references for foundational concepts used throughout the lecture. Skip or jump back as needed.

Version control with Git.

Track changes to your code over time. Work safely in teams. Never lose work again.



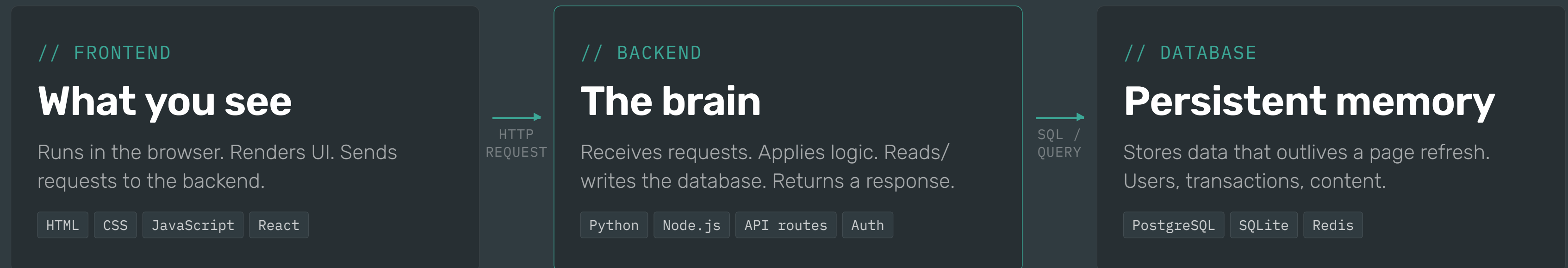
`git clone <url> – copy a remote repo locally`

`git pull – sync your local with remote`

`git branch – create separate lines of work`

Frontend vs. backend vs. database.

Every web application is three layers communicating over HTTP.



Databases: how data is stored.

Two main paradigms — pick based on structure and scale.

// SQL · RELATIONAL

Structured tables

Data in rows and columns with defined relationships. Best for financial data, user records — anything where structure matters.

id	name	email	balance
1	Yazan	y@josa.jo	1284.50
2	Ahmad	a@josa.jo	842.00
3	Sara	s@josa.jo	3100.75

```
SELECT * FROM users WHERE balance > 1000
```

// NOSQL · DOCUMENT

Flexible documents

Data as JSON-like objects — no fixed schema. Best for varied, nested, or rapidly evolving structures.

```
{
  "user_id": "u_001",
  "name": "Yazan",
  "transactions": [
    { "cat": "food", "amount": -42.20 }
  ]
}
```

MongoDB

Firestore

DynamoDB

HTTP: how the web communicates.

Every frontend-to-backend interaction is an HTTP request-response cycle.

// CLIENT

Browser / App

Sends a request with a method, URL, headers, and optional body.

GET POST PUT DELETE

→ REQUEST

GET /api/transactions
Authorization: Bearer <token>
Accept: application/json

← RESPONSE

200 OK
Content-Type: application/json
[{"id":1, "amount":-42.20, "cat":"food" }, ...]

// SERVER

Backend API

Receives the request, runs logic, queries the DB, returns a status code and body.

200 OK — success

401 Unauthorized

404 Not Found

500 Server Error